

Assignment 4: Multi-Client Chat Server using TCP

Objective

Develop a multi-client chat application using TCP sockets where multiple clients can communicate through a centralized server. The assignment aims to introduce concurrent server design, client management, message broadcasting, and performance evaluation.

Network Setup

Use Mininet with one server and three clients.

```
sudo mn --topo single,4
```

Topology:

h1 = Chat Server

h2 = Client A

h3 = Client B

h4 = Client C

Verify connectivity using:

nodes

net

pingall

Task 1: Multi-Client Chat Server (30 Marks)

Develop a TCP chat server that listens on port 5000, accepts multiple client connections simultaneously, uses threads to handle clients, maintains active client information, and logs connection/disconnection events.

Server log format:

timestamp,EVENT,username,client_ip

Examples:

10:00:01,CONNECTED,Rahul,10.0.0.2

10:15:20,DISCONNECTED,Rahul,10.0.0.2

Task 2: User Registration and Broadcast Messaging (25 Marks)

Clients must register using a username. The server broadcasts messages to all connected clients.

Example:

Enter Username:

Bishal

Client message:

Hello Everyone

Received:

[Bishal] Hello Everyone

Store all chat messages in chat_log.txt using:
timestamp,username,message

Task 3: Performance Measurement (15 Marks)

Conduct experiments using 1, 2, and 3 clients. Each client sends 20 messages.

Measure:

- Average message delivery time
- Total messages delivered
- Throughput

Store results in performance_results.csv

Columns:

clients,total_messages,avg_delivery_time_ms,throughput_msgs_per_sec

Task 4: Wireshark Verification (15 Marks)

Capture traffic using Wireshark.

Filter:

tcp.port == 5000

Capture screenshots of:

- TCP Three-Way Handshake
- Chat Message Packet
- Server Broadcast Packet
- TCP Connection Termination

Required files:

tcp_handshake.png

chat_message.png

broadcast_message.png

connection_close.png

Task 5: Graph Generation (10 Marks)

Generate:

1. Number of Clients vs Average Delivery Time (clients_vs_delay.png)
2. Number of Clients vs Throughput (clients_vs_throughput.png)

Task 6: Reflection Questions (5 Marks)

Answer in not more than 300 words total:

1. Why is a multi-threaded server preferred over a single-threaded server?
2. What challenges arise when multiple clients communicate simultaneously?
3. How does TCP ensure reliable delivery of chat messages?
4. What information from Wireshark helped verify the application behavior?

System and Execution Details

Include outputs of:

```
uname -a  
python3 --version  
ip addr
```

Execution commands:

```
h1 python3 server.py  
h2 python3 client.py  
h3 python3 client.py  
h4 python3 client.py
```

Submission Guidelines

Submit:

ROLLNO_NAME_CHATSERVER.zip

Folder Structure:

```
ROLLNO_NAME_CHATSERVER/  
├── server.py  
├── client.py  
├── chat_log.txt  
├── performance_results.csv  
├── graphs/  
├── screenshots/  
└── report.pdf
```

Report Format

Objective
Network Topology
Program Design
Experimental Results
Wireshark Verification
Graph Analysis
Reflection Answers
Conclusion

Important Notes

The application must run inside Mininet. All screenshots must be from the student's own experiment. CSV files and graphs must be generated from actual execution results. Students must be prepared to demonstrate the application during evaluation. Submissions without logs, graphs, screenshots, or Wireshark evidence will not be evaluated.

Evaluation Criteria

Component	Marks
Multi-client server implementation	30
User registration and broadcast messaging	25
Performance measurement	15
Wireshark verification	15
Graph generation	10
Reflection and report quality	5

Total Marks: 100